

Der React `useReducer()` Hook

Ein tiefer Einblick in fortgeschrittenes State-Management für komplexe React-Komponenten

1. Was ist `useReducer()` ?

Der `useReducer` Hook ist eine eingebaute Funktion in React, die eine Alternative zu `useState` für das Zustandsmanagement (State-Management) in Komponenten bietet. Er basiert auf dem Prinzip des "Reducers", das aus Bibliotheken wie Redux oder der funktionalen Programmierung bekannt ist.

Während `useState` ideal für einfache, unabhängige Zustände ist, eignet sich `useReducer` hervorragend für komplexere Zustandslogiken, bei denen der nächste Zustand vom vorherigen abhängt oder mehrere Werte gleichzeitig aktualisiert werden müssen.

Die Kern-Syntax

```
const [state, dispatch] = useReducer(reducer, initialState, init);
```

Der Hook akzeptiert primär zwei Argumente und gibt ein Array mit zwei Elementen zurück:

- **reducer** : Eine reine Funktion (Pure Function), die den aktuellen State und eine **action** entgegennimmt und den neuen State zurückgibt: `(state, action) => newState`.
- **initialArgument** : Der Startzustand (Initial State) der Komponente.
- **state** : Der aktuelle Zustand.
- **dispatch** : Eine Funktion, mit der man eine Aktion "auslösen" (dispatchen) kann, um eine Zustandsänderung zu triggern.

Wichtig: Pure Functions

Der Reducer muss immer eine **reine Funktion** sein. Er darf keine Seiteneffekte (wie API-Abrufe) ausführen und darf den bestehenden State nicht direkt mutieren, sondern muss immer ein neues Objekt bzw. einen neuen Wert zurückgeben.

2. Wann sollte man `useReducer()` anstelle von `useState()` benutzen?

Die Entscheidung zwischen den beiden Hooks hängt von der Komplexität Ihres Vorhabens ab. Hier sind die Hauptgründe für den Einsatz von `useReducer` :

Kriterium	Verwende <code>useState</code>	Verwende <code>useReducer</code>
Zustandsart	Einfache Werte (Strings, Numbers, einfache Booleans)	Komplexe Objekte, verschachtelte Arrays, miteinander verknüpfte Daten
Logik-Abhängigkeit	Der nächste State hängt nicht vom alten State ab	Der nächste State basiert stark auf Berechnungen des vorherigen States
Anzahl der Aktionen	Wenige, isolierte Updates (z.B. ein Toggle-Switch)	Viele verschiedene Aktionen, die den State auf unterschiedliche Weise manipulieren
Testbarkeit	Integrierte Logik in der UI, schwerer isoliert zu testen	Die Reducer-Funktion kann komplett außerhalb der Komponente getestet werden

3. Praktische Beispiele

Beispiel 1: Der klassische Zähler (Einfaches Beispiel)

Dieses Beispiel veranschaulicht die Funktionsweise mit einfachen mathematischen Operationen, bei denen klar wird, wie `dispatch` und der Reducer interagieren.

```

import React, { useReducer } from 'react';

// 1. Initialer Zustand definieren
const initialState = { count: 0 };

// 2. Reducer Funktion schreiben
function counterReducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return initialState;
    default:
      throw new Error('Unbekannte Aktion!');
  }
}

export default function CounterComponent() {
  // 3. Hook in der Komponente initialisieren
  const [state, dispatch] = useReducer(counterReducer, initialState);

  return (
    <div>
      <p>Anzahl: {state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
      <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
    </div>
  );
}

```

Beispiel 2: Ein komplexes Formular- und API-Handling (Realistisches Szenario)

Stellen Sie sich vor, Sie laden Daten aus einer API. Sie müssen den Ladestatus (`loading`), Fehler (`error`) und die eigentlichen Daten (`data`) verwalten. Mit `useState` müssten Sie drei separate States synchron halten. Mit `useReducer` fassen Sie dies kompakt zusammen:

```

import React, { useReducer, useEffect } from 'react';

const fetchInitialState = {
  loading: false,
  error: null,
  data: []
};

function fetchReducer(state, action) {
  switch (action.type) {
    case 'FETCH_START':
      return { ...state, loading: true, error: null };
    case 'FETCH_SUCCESS':
      return { ...state, loading: false, data: action.payload };
    case 'FETCH_ERROR':
      return { ...state, loading: false, error: action.payload };
    default:
      return state;
  }
}

export default function DataFetchComponent() {
  const [state, dispatch] = useReducer(fetchReducer, fetchInitialState);

  useEffect(() => {
    dispatch({ type: 'FETCH_START' });
    fetch('https://api.example.com/items')
      .then(res => res.json())
      .then(data => {
        dispatch({ type: 'FETCH_SUCCESS', payload: data });
      })
      .catch(err => {
        dispatch({ type: 'FETCH_ERROR', payload: err.message });
      });
  }, []);

  if (state.loading) return <p>Lädt Daten...</p>;
  if (state.error) return <p>Fehler: {state.error}</p>;

  return (
    <ul>
      {state.data.map(item => <li key={item.id}>{item.name}</li>)}
    </ul>
  );
}

```

4. Zusammenfassung

`useReducer()` nimmt Ihnen nicht die Arbeit ab, strukturiert Ihren Code bei komplexer Logik jedoch deutlich sauberer. Dadurch, dass die gesamte Update-Logik an einem zentralen Ort (der Reducer-Funktion) gebündelt wird, bleibt die eigentliche UI-Komponente frei von unübersichtlichen Zustandsberechnungen.

- Verwenden Sie `useState` für unkomplizierte, unabhängige UI-Zustände.
- Verwenden Sie `useReducer`, sobald sich logische Abhängigkeiten bilden oder komplexe Strukturen im State vorliegen.